Relatório — Extensão do OpenSSL 3 com SPECK128/128 e Trivium

Mestrado em Cibersegurança — Criptografia Aplicada

Primeiro trabalho prático. Ano letivo 2026/2027.

Instituição: Departamento de Eletrónica, Telecomunicações e Informática, Universidade de Aveiro

Autores: Ricardo Duarte (n.º 138628), Fábio Oliveira (n.º 138627)

Docentes: Professor André Zúquete e Professor Tomás Oliveira e Silva

Data de entrega do relatório: 08/11/2026

1. Resumo

Este trabalho, realizado no âmbito da UC de Criptografia Aplicada e na linguagem C, integra duas cifras simétricas num provider do OpenSSL 3: a cifra por blocos SPECK-128 e a cifra de fluxo Trivium. O provider, denominado ca, é compilado como um módulo independente, ca.so, que o executável openssl instalado no sistema carrega dinamicamente. O SPECK é disponibilizado nos modos ECB, CBC e CTR como pedido nos requisitos mínimos do projeto. A implementação permite listar os algoritmos, cifrar e decifrar ficheiros e executar medições através de ***openssl speed***.

Ficheiros de implementação dos métodos de cifra estão separados do código que implementa a interface EVP/provider. O projeto trata a inicialização, os parâmetros, a continuidade entre chamadas, o padding e os limites do estado. A validação é organizada em vetores de resposta conhecida, testes dos modos de operação, testes de integração e recuperação de ficheiros.

2. Seleção dos algoritmos e referências

2.1. SPECK128/128

Foi escolhida a variante SPECK128/128, com bloco e chave de 128 bits. A indicação SPECK128/128 é relevante porque outras variantes da família têm parâmetros diferentes. A estrutura usa adição modular, rotações e XOR, operações que podem ser traduzidas diretamente para C com inteiros de 64 bits. A escolha permite uma implementação curta e legível.

O ficheiro speck128.c implementa as funções de ronda, ronda inversa, expansão de chave e conversões de bytes. As operações das macros de referência ER64 e DR64 foram separadas em

funções com nomes explícitos. A função ***speck128_init*** segue a organização de ***Speck128128KeySchedule[2].***

2.2. Trivium

O Trivium é uma cifra de fluxo que usa uma chave de 80 bits, um IV de 80 bits e um estado interno de 288 bits, dividido em três registos de 93, 84 e 111 bits. Na implementação, este estado é guardado num array em que cada elemento representa um bit, com o valor zero ou um.

A função ***trivium_init*** coloca a chave e o IV nas posições iniciais dos dois primeiros registos. As restantes posições ficam a zero, exceto as três últimas, que ficam a um. Depois, executa 1 152 atualizações do estado, descartando os bits de saída. Estas atualizações combinam os bits da chave e do IV com os restantes bits do estado antes de começar a cifragem.

Cada chamada a ***trivium_clock*** calcula um bit de saída e três novos bits através de operações XOR e AND entre posições específicas do estado. Em seguida, desloca os bits guardados e introduz os três novos bits no início dos respetivos registos. A função ***trivium_next_byte*** repete este processo oito vezes para formar um byte de keystream.

A função ***trivium_crypt*** aplica XOR entre cada byte da entrada e um byte de keystream. Para decifrar, repete-se a inicialização com a mesma chave e IV e aplica-se a mesma operação ao texto cifrado. O contexto conserva o estado entre chamadas, permitindo processar um ficheiro por fragmentos sem reiniciar a sequência.

2.3. Elegibilidade das cifras escolhidas

A família SPECK e a cifra Trivium são mencionadas no enunciado como possíveis candidatas para o projeto. A escolha de SPECK128/128 e Trivium partiu dessa indicação, tendo também em conta a simplicidade das operações e a disponibilidade de documentação e vetores de teste.

A verificação da sua presença no OpenSSL limitou-se ao comando ***openssl list -cipher-algorithms***. Na instalação utilizada, as cifras escolhidas não apareceram na listagem consultada. O alcance desta verificação restringe-se à versão e aos providers disponíveis nesse ambiente, não foi realizada uma pesquisa do histórico de versões nem uma análise independente do estado de segurança das cifras.

3. Arquitetura e organização do projeto

3.1. Papel de cada componente

O projeto é usado através do comando openssl instalado no sistema. Ao indicar *-provider-path . -provider ca*, o OpenSSL carrega o módulo ca.so e passa a ter acesso às cifras que implementámos.

O comando trata dos argumentos e da leitura e escrita dos ficheiros. As operações criptográficas são feitas através de EVP, que permite usar diferentes algoritmos da mesma forma. O nosso provider fornece as funções que EVP vai chamar para inicializar a cifra, processar os dados e terminar a operação.

Quando o módulo é carregado, o OpenSSL chama ***OSSL_provider_init***, definida em **provider.c**. A partir daí, pode consultar os dados do provider e pedir a lista de algoritmos. A função ***provider_query*** responde ao pedido **OSSL_OP_CIPHER** com os nomes das cifras e as respetivas

tabelas de funções. Estas tabelas, do tipo **OSSL_DISPATCH**, indicam qual é a função do nosso código que corresponde a cada operação esperada pelo OpenSSL.

Por exemplo, ao cifrar um ficheiro com **SPECK-CBC**, o OpenSSL lê parte do ficheiro e entrega esses bytes ao provider através de EVP. O código em **provider_cipher.c** trata da divisão em blocos e chama **speck128.c** para cifrar cada bloco. Os bytes produzidos são depois escritos pelo OpenSSL no ficheiro de saída.

Os ficheiros do projeto estão organizados da seguinte forma:

- **provider.c** contém a entrada do módulo, os dados do provider e o registo das cifras.
- **provider_cipher.c** faz a ligação ao OpenSSL e trata dos modos, dos parâmetros, dos dados pendentes e do padding.
- **speck128.c** contém a expansão da chave e a cifragem e decifragem de blocos SPECK.
- **trivium.c** contém a inicialização do Trivium, a atualização do estado e a geração e aplicação do keystream, fazendo assim a cifragem e decifragem.
- **CMakeLists.txt** define como compilar e ligar o módulo.

Apesar de estarem na mesma diretoria, os ficheiros das cifras podem ser usados separadamente do provider. As cifras **SPECK** e **Trivium** trabalham com arrays de bytes e não precisam de conhecer os comandos ou os ficheiros usados pelo OpenSSL.

3.2. Cifras Disponíveis

O provider regista quatro nomes, que são usados diretamente nos comandos:

- **SPECK128-128-ECB:** chave de 16 bytes, sem IV e blocos de 16 bytes.
- **SPECK128-128-CBC:** chave de 16 bytes, IV de 16 bytes e blocos de 16 bytes.
- **SPECK128-128-CTR:** chave de 16 bytes e IV de 16 bytes.
- **TRIVIUM**: chave de 10 bytes e IV de 10 bytes.

Todos permitem cifrar e decifrar. Em **ECB** e **CBC**, padding PKCS#7 está ativo por omissão, podendo ser desligado com ***-nopad.*** Nesse caso, o tamanho dos dados deve ser múltiplo de 16 bytes.

CTR e Trivium processam qualquer número de bytes e não acrescentam padding. Por esse motivo, ambos anunciam ao OpenSSL uma unidade de processamento de 1 byte. No caso de CTR, o SPECK continua a cifrar blocos de 16 bytes para produzir o keystream, que vai sendo consumido conforme necessário.

3.3. Estado de uma operação

O OpenSSL pode entregar um ficheiro em vários fragmentos. É preciso guardar informação entre chamadas para que o processamento continue do ponto onde ficou. Essa informação está na estrutura ***cipher_ctx***, referida no código pela variável ***ctx***.

No SPECK, o contexto guarda as subchaves calculadas na inicialização. No Trivium, guarda o estado dos três registos e a quantidade de keystream já utilizada. Estes dados ficam no campo ***core***, que contém a estrutura da cifra selecionada.

O contexto guarda ainda a chave em ***key*** e o IV inicial em ***original_iv***. O campo **iv** é usado em CBC, onde guarda o bloco cifrado necessário para continuar o encadeamento e em CTR onde guarda o contador.

Os campos ***pending*** e ***pending_length*** servem para os dados que ainda não podem ser processados em ECB e CBC. Se uma chamada entregar apenas 10 bytes, por exemplo, estes ficam guardados até chegar o suficiente para completar um bloco ou até à finalização.

Em CTR, ***keystream*** guarda os 16 bytes produzidos pela cifragem do contador, e ***stream_position*** indica quantos já foram usados. Assim, uma chamada pode consumir apenas parte desse bloco e deixar o resto para a seguinte.

Os restantes campos controlam a operação. ***padding*** indica se há padding e ***encrypting*** indica a direção. ***key_set***, ***iv_set*** e ***initialized*** permitem verificar se a cifra está pronta. ***started***, ***finished*** e ***failed*** registam se o processamento começou, terminou ou ficou num estado de erro. ***counter_exhausted*** assinala que CTR já não pode gerar outro bloco de keystream com esse contador.

```c
static const cipher_spec ecb_spec = {EVP_CIPH_ECB_MODE, 16, 0, 16};
static const cipher_spec cbc_spec = {EVP_CIPH_CBC_MODE, 16, 16, 16};
static const cipher_spec ctr_spec = {EVP_CIPH_CTR_MODE, 16, 16, 1};
static const cipher_spec trivium_spec = {EVP_CIPH_STREAM_CIPHER, 10, 10, 1};

typedef struct {
    const cipher_spec *spec;
    union {
        speck128_ctx speck;
        trivium_ctx trivium;
    } core;
    uint8_t key[16];
    uint8_t original_iv[16];
    uint8_t iv[16];
    uint8_t pending[16];
    uint8_t keystream[16];
    size_t pending_length;
    size_t stream_position;
    unsigned int padding; //tudo a partir daqui ou e 0 ou 1
    int encrypting;
    int key_set;
    int iv_set;
    int initialized;
    int started;
    int finished;
    int failed;
    int counter_exhausted;
} cipher_ctx;
```

3.4. Funções usadas pelo OpenSSL

newctx cria o contexto com os parâmetros da cifra escolhida. *dupctx* permite copiá-lo a meio de uma operação, incluindo os dados pendentes e a posição atual. Como cada cópia fica com o seu próprio estado, as duas podem continuar a ser usadas separadamente. No fim, *freectx* limpa e liberta a memória.

encrypt_init e ***decrypt_init*** preparam a cifragem ou a decifragem. Ambas chamam ***cipher_init***, que verifica a chave e o IV fornecidos, repõe os buffers e inicializa a cifra. A diferença entre as duas chamadas é a direção da operação.

update recebe os fragmentos de entrada. A função ***cipher_update*** escolhe o processamento adequado: ***block_update*** para ECB e CBC, ***ctr_update*** para CTR ou ***trivium_crypt*** para Trivium. Também informa o OpenSSL de quantos bytes foram produzidos.

final trata do fim da entrada. Em ECB e CBC, acrescenta o padding na cifragem ou verifica e retira esse padding na decifragem. Com ***-nopad***, verifica se ficou algum bloco incompleto. Em CTR e Trivium, termina a operação sem acrescentar bytes.

As funções de parâmetros permitem ao OpenSSL consultar e configurar a implementação. ***get_params*** devolve as características fixas da cifra, como o modo e os tamanhos de chave, IV e bloco. ***get_ctx_params*** consulta dados da operação atual. ***set_ctx_params*** verifica os comprimentos pedidos e permite configurar o padding antes de começar a processar dados.

Existem ainda ***gettable_params***, ***gettable_ctx_params*** e ***settable_ctx_params***. Estas funções indicam quais os parâmetros que as anteriores aceitam e os respetivos tipos.

Além destas treze funções exigidas pelo enunciado, foi incluída ***cipher_raw*** para suportar a chamada direta EVP_Cipher. Esta função processa os dados sem acrescentar padding e exige blocos completos nos modos ECB e CBC.

4. Implementação do SPECK128/128 e dos modos

4.1. Representação, ronda e expansão de chave

Um bloco de 16 bytes é convertido em duas palavras de 64 bits. ***load_le64*** interpreta os primeiros oito bytes como *y* e os oito seguintes como *x*, em little-endian. ***store_le64*** faz a conversão inversa. A montagem explícita das palavras evita depender da ordem de bytes da máquina ou de acessos a uint64_t através de endereços desalinhados.

Na função ***speck_round*****,** *x* é rodado oito posições à direita, somado a *y* e combinado por XOR com a subchave. Depois, *y* é rodado três posições à esquerda e combinado por XOR com o novo *x*. Como as palavras são uint64_t, a soma é calculada módulo 2^{64}. A função ***speck_inverse_round*** desfaz estas operações pela ordem inversa, usando subtração modular e rotações opostas.

speck128_init lê a chave em duas palavras, a e b. Guarda a como primeira subchave e reutiliza a transformação da ronda, com o índice i, para obter as restantes. As 32 subchaves ficam em round_keys. ***speck128_cipher_block*** aplica-as pela ordem normal, ***speck128_decipher_block*** aplica

as rondas inversas com as subchaves pela ordem inversa. A expansão é feita na inicialização, não para cada bloco do ficheiro.

4.2. ECB

Em ECB, cada bloco é tratado de forma independente:

- C[i] = E(K, P[i])
- P[i] = D(K, C[i])

process_block chama diretamente o núcleo e não usa IV. Blocos de texto em claro iguais, com a mesma chave, originam blocos cifrados iguais. Este comportamento explica a limitação do modo perante dados com padrões.

4.3. CBC

Em CBC, define-se C[0] = IV. A cifragem calcula C[i] = E(K, P[i] XOR C[i−1]). A decifragem calcula P[i] = D(K, C[i]) XOR C[i−1]. ***process_block*** implementa estas duas sequências e atualiza IV com o bloco cifrado necessário para a chamada seguinte. O bloco de entrada é preservado durante a decifragem para permitir também a operação no mesmo buffer.

4.4. CTR

CTR trata os 16 bytes do IV como um único contador de 128 bits em big-endian. O contador atual é cifrado com SPECK, o resultado é combinado por XOR com os dados e o contador é incrementado a partir do byte 15, propagando o transporte para os anteriores. O primeiro bloco usa exatamente o valor inicial fornecido. Esta convenção do contador é independente da conversão little-endian feita dentro do núcleo SPECK.

ctr_update conserva os bytes do bloco de keystream que ainda não foram usados. Por isso, chamadas com fragmentos diferentes continuam a mesma sequência. A decifragem usa a mesma operação, porque aplicar duas vezes o XOR com o mesmo fluxo recupera a entrada. ***ctr_can_process*** verifica antecipadamente se um pedido ultrapassaria o espaço de contadores. O último bloco, com contador todo a um, pode ser consumido, não é permitido gerar outro bloco após a volta do contador.

4.5. Comprimentos arbitrários e padding

block_update acumula bytes até formar blocos completos. Em decifragem com padding, retém o último bloco para ***cipher_final*** verificar e retirar o padding. Se faltarem *p* bytes para completar um bloco, a cifragem acrescenta *p* bytes com o valor *p*. Para entradas já múltiplas de 16, incluindo a vazia, acrescenta um bloco completo com dezasseis bytes com o valor 0x10.

Uma entrada de 17 bytes ocupa dois blocos em ECB e CBC. Com o padding, o resultado cifrado tem 32 bytes. Em CTR e Trivium, o resultado mantém os 17 bytes, porque estes modos não precisam de completar blocos.

Ao usar -nopad em ECB ou CBC, o tamanho da entrada tem de ser múltiplo de 16 bytes. Se o último bloco ficar incompleto, a operação termina com erro. Na decifragem, também é devolvido um erro se o texto cifrado tiver um bloco incompleto ou, quando o padding está ativo, se este for inválido.

5. Implementação do Trivium

5.1. Estado e inicialização

trivium_ctx contém state[288], um array de uint8_t cujos elementos representam bits, e ***bytes_generated***, que contabiliza os bytes de fluxo já utilizados. Os três registos ocupam state[0..92], state[93..176] e state[177..287], com comprimentos de 93, 84 e 111 bits respetivamente.

trivium_init começa por limpar o contexto. Coloca os bits da chave nas primeiras 80 posições e os do IV a partir da posição 93. As posições 285, 286 e 287 recebem o valor um, as restantes posições não ocupadas ficam a zero. Nesta implementação, os bytes da chave e do IV são lidos do índice 9 para o índice 0, do bit mais significativo para o menos significativo em cada byte. Por exemplo, state[0] recebe o bit 7 de key[9]**.**

Logo após a inicialização, o estado contém os bits da chave, os bits do IV e várias posições fixas a 0 ou 1. Nesse momento, o primeiro bit de saída seria apenas o XOR entre um bit da chave, um bit do IV e o valor 1. Conhecendo o IV e esse bit de saída, seria possível descobrir diretamente o bit da chave.

As 1152 chamadas a ***trivium_clock*** servem para transformar esse estado inicial antes de usar a saída. Em cada chamada, são calculados três novos bits com operações XOR e AND sobre bits do estado. Esses novos bits entram nos registos e, à medida que são deslocados, passam a participar nos cálculos seguintes. Assim, a saída passa a depender de combinações sucessivas dos bits da chave e do IV. Os bits produzidos durante esta preparação são descartados. O número de 1152 passos faz parte da definição do Trivium.

Na decifragem, repete-se esta preparação com a mesma chave e IV para reproduzir o mesmo keystream. Durante o processamento dos fragmentos de um ficheiro, conserva-se o estado e continua-se a sequência.

5.2. Atualização e geração de bytes

trivium_clock calcula três valores temporários por XOR de posições do estado. O XOR desses valores fornece o bit de saída. Cada valor temporário recebe depois um termo AND entre dois bits e um XOR com um bit de outro registo, formando os três bits de realimentação. Estas operações são calculadas antes de alterar o array.

O ciclo de deslocamento percorre o array de trás para a frente, para não substituir um valor antes de o copiar. Em seguida, as posições 0, 93 e 177 recebem os novos bits. ***trivium_next_byte*** chama este processo oito vezes, o primeiro bit gerado ocupa o bit 0 do byte resultante. A numeração da especificação começa em 1, enquanto os índices do array C começam em 0.

trivium_generate devolve apenas o keystream. ***trivium_crypt*** combina cada byte desse fluxo por XOR com um byte da entrada. A mesma função cifra e decifra:

- C = P XOR Z

- P = C XOR Z

Os dados de entrada não alimentam o estado do Trivium, o estado evolui segundo a chave, o IV e o número de passos já executados.

5.3. Continuidade, limites e custo

Chamadas sucessivas continuam a sequência a partir do estado atual. A reinicialização repõe a sequência inicial, a cópia do contexto permite duas continuações independentes. Pedidos vazios não geram bytes. O limite implementado é 2^{64} bits de keystream, equivalente a 2^{61} bytes, por inicialização com um par chave/IV. Um pedido que o ultrapasse é recusado antes de escrever a saída ou avançar o estado.

Cada posição do array guarda apenas um zero ou um, mas ocupa um byte inteiro. Assim, os 288 bits do estado ocupam 288 bytes de memória. Se fossem guardados oito bits em cada byte, seriam necessários apenas 36 bytes. Usámos uma posição por bit para facilitar o acesso aos bits e a leitura do código.

Esta escolha também tem um custo no processamento. Para gerar cada bit, o ciclo desloca 287 posições do array. Como cada byte de saída exige oito passos, são feitas 2296 cópias, além dos cálculos dos novos bits. Este trabalho deve ser tido em conta na análise das velocidades medidas, que correspondem à nossa implementação do Trivium.

6. Gestão de estado, erros e buffers

EVP pode configurar o algoritmo, a chave e o IV em chamadas separadas. ***cipher_init*** aceita esta inicialização por etapas, e ***cipher_ready*** impede o processamento enquanto faltarem dados obrigatórios. Quando uma reinicialização não fornece uma nova chave ou um novo IV, os valores já configurados podem ser conservados, o IV de trabalho e os buffers são repostos para o início da nova operação.

Os tamanhos e tipos dos parâmetros são validados. Os comprimentos são fixos para cada algoritmo, configurar um comprimento diferente não seleciona outra variante. A inicialização usa uma cópia temporária do contexto antes de confirmar as alterações. A finalização com erro de padding marca a operação como falhada, e uma operação terminada ou falhada deve ser *reset* antes de voltar a receber dados.

Os buffers de saída são verificados antes do processamento correspondente. Em CBC com padding, a verificação do último bloco usa uma cópia do contexto, permitindo repetir a finalização se o espaço de saída for insuficiente. Os núcleos admitem entrada e saída exatamente no mesmo endereço. Em ECB/CBC, se já existirem bytes pendentes, o adaptador faz uma cópia temporária da entrada dessa chamada para não sobrescrever dados ainda por consumir. A sobreposição parcial de buffers não é suportada.

cipher_error acrescenta informação à pilha de erros do OpenSSL. A libertação do contexto limpa a sua memória e vários buffers temporários são limpos explicitamente.

7. Compilação e utilização

7.1. Dependências e compilação

São necessários um compilador C, CMake 3.16 ou superior, uma ferramenta de construção como Make, OpenSSL 3, os respetivos headers de desenvolvimento e libcrypto. Na diretoria do projeto executam-se, por esta ordem:

```
Shell
cmake .
cmake --build .
```

O primeiro comando configura a construção, o segundo compila. O CMake gera a biblioteca estática libcipher_cores.a a partir dos núcleos e integra-a no módulo ca.so, juntamente com o adaptador e a ligação a OpenSSL::Crypto. A biblioteca estática não é um segundo provider a carregar. POSITION_INDEPENDENT_CODE permite incluir os objetos dessa biblioteca no módulo dinâmico. PREFIX vazio faz com que o nome final seja ca.so, em vez de libca.so.

7.2. Carregamento e listagem

```
Shell
openssl list -provider-path . -provider ca -providers
openssl list -provider-path . -provider ca -cipher-algorithms -propquery
provider=ca
```

Espera-se encontrar o provider ativo e os quatro nomes indicados na secção 3.2. **-provider-path** . aponta para a diretoria do módulo e **-provider ca** pede o seu carregamento. A propriedade **provider=ca** restringe a listagem às implementações do projeto.

7.3. Exemplo completo com SPECK-CBC

Os comandos seguintes usam um ficheiro de teste *input.txt* como entrada e um ficheiro de saída *recovered.txt*. A chave e o IV são valores de teste em hexadecimal. Cada par de caracteres representa um byte.

```
Shell
openssl enc -e -provider-path . -provider ca -SPECK128-128-CBC -K
000102030405060708090a0b0c0d0e0f -iv 101112131415161718191a1b1c1d1e1f -in
input.txt -out encrypted.txt

openssl enc -d -provider-path . -provider ca -SPECK128-128-CBC -K
000102030405060708090a0b0c0d0e0f -iv 101112131415161718191a1b1c1d1e1f -in
encrypted.txt -out recovered.txt
```

O comando a seguir valida se o texto recuperado coincide com o introduzido:
cmp input.txt recovered.txt

Para ECB, substitui-se o nome por **SPECK128-128-ECB** e retira-se **-iv**. Para CTR, usa-se **SPECK128-128-CTR** com chave e IV de 16 bytes. A opção **-nopad** aplica-se aos testes de ECB/CBC com comprimentos múltiplos de 16, CTR não precisa de padding. O comando **cmp** termina com código zero quando os ficheiros são iguais e, nesse caso, não imprime nada.

7.4. Exemplo completo com Trivium

```Shell
openssl enc -provider-path . -provider ca -TRIVIUM -K 00010203040506070809 -iv
10111213141516171819 -in input.txt -out encrypted.txt

openssl enc -d -provider-path . -provider ca -TRIVIUM -K 00010203040506070809
-iv 10111213141516171819 -in encrypted.txt -out recovered.txt

cmp input.txt  recovered.txt
```

Estes exemplos fornecem 20 caracteres hexadecimais para a chave e outros 20 para o IV, correspondentes aos 10 bytes exigidos pelo Trivium. Acrescentar -a às operações de cifragem e decifragem permite guardar e ler o texto cifrado em Base64. A codificação é tratada pelo OpenSSL, o provider continua a receber bytes. Sem -a, a saída é binária, mesmo que o nome do ficheiro termine em .txt. [8]

7.5. Sequência de execução de um ficheiro

Ao executar o comando, o OpenSSL lê as opções fornecidas e carrega o módulo ca.so, chamando ***OSSL_provider_init***. Depois, consulta as cifras registadas através de OSSL_OP_CIPHER e seleciona a indicada no comando. A tabela dessa cifra identifica as funções que serão chamadas durante a operação. Os valores hexadecimais de **-K** e **-iv** são convertidos em bytes e usados para preparar o contexto com a chave e o IV.

O OpenSSL lê o ficheiro por partes e entrega cada uma a ***cipher_update***. Esta função encaminha os dados para o processamento escolhido e devolve os bytes produzidos, que o OpenSSL escreve no ficheiro de saída. O contexto é mantido entre chamadas para conservar os dados pendentes e o estado da cifra.

Quando chega ao fim do ficheiro, o OpenSSL chama ***cipher_final***. Em ECB e CBC com padding, esta função acrescenta o preenchimento na cifragem ou verifica-o e retira-o na decifragem. Depois, o contexto é libertado e os ficheiros são fechados.

A opção **-d** escolhe a decifragem. Em ECB e CBC, são usadas as rondas inversas do SPECK. Em CTR e Trivium, é gerado novamente o mesmo keystream, a partir da mesma chave e IV, e aplicado XOR para recuperar os dados

8. Validação funcional

8.1. Validação SPECK128/128

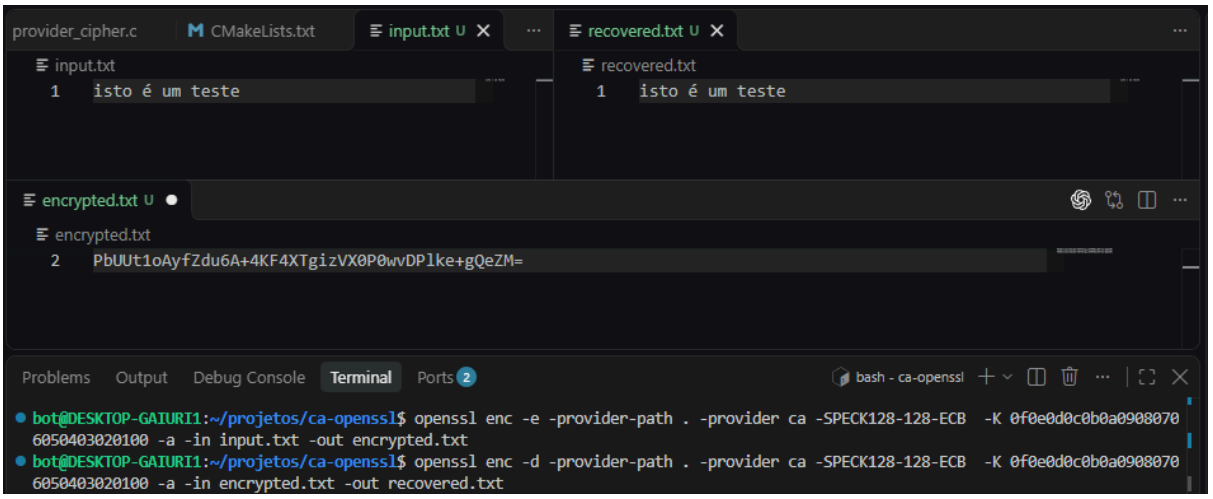
A cifragem SPECK128/128 foi comparada com a implementação disponível no site SPECK Encryption. Foi usado o mesmo texto e a mesma chave no site e no projeto:

Texto em claro: isto é um teste
Chave(Key type: custom): 0f0e0d0c0b0a09080706050403020100
Modo: ECB
Padding: PKCS#7
Formato da saída: Base64

O resultado cifrado foi:

PbUUt1oAyfZdu6A+4KF4XTgizVX0P0wvDPlke+gQeZM=

O resultado do projeto coincidiu com o da implementação externa. Esta comparação verifica a cifragem através do comando OpenSSL, incluindo a ligação ao provider e o preenchimento do bloco. Não foi feita uma comparação individual das subchaves.



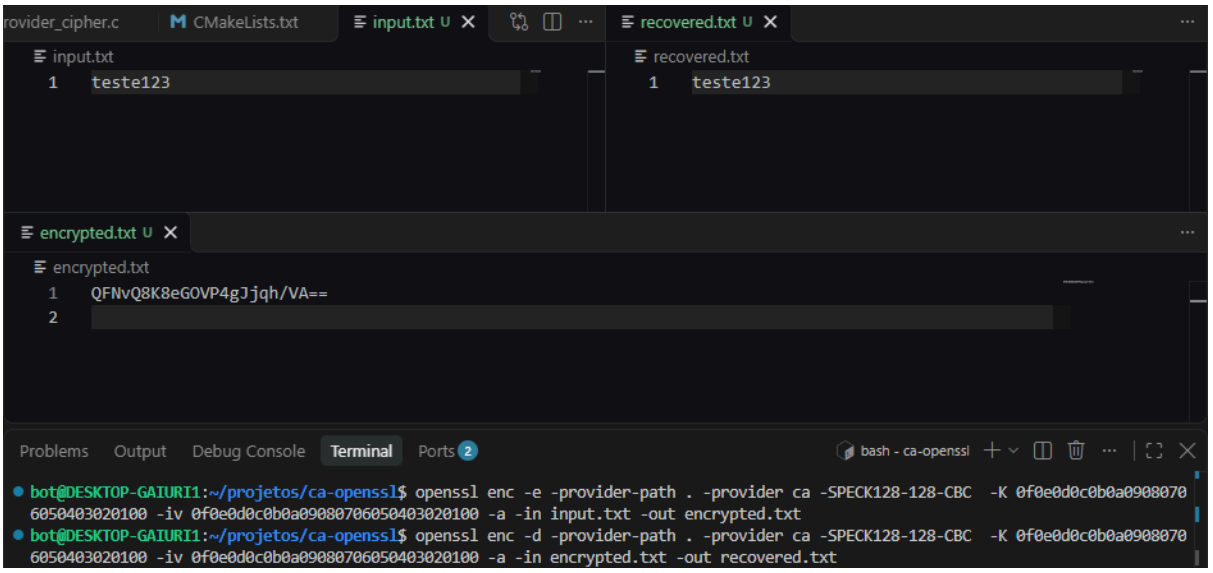
8.2. Respostas conhecidas dos modos SPECK

O teste anterior foi realizado em ECB, com padding PKCS#7. Como o texto tinha 16 bytes (UTF-8) e o SPECK trabalha com blocos de 16 bytes, foi acrescentado um bloco completo de padding antes da cifragem.

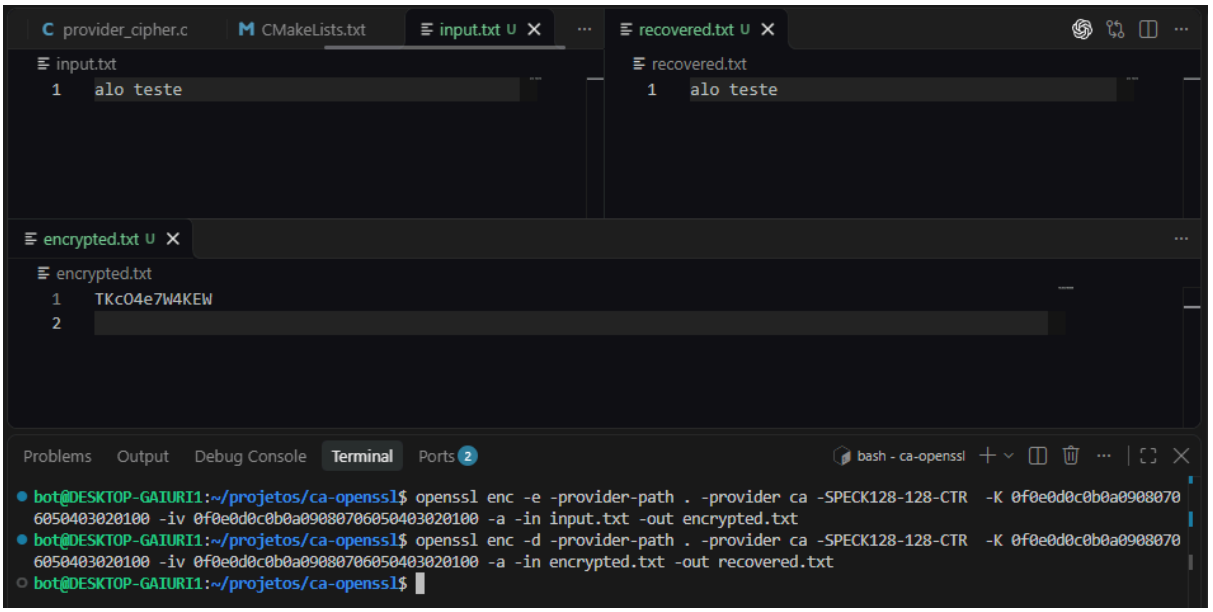
Para completar a comparação dos modos CBC e CTR, devem ser usados o mesmo texto, a mesma chave e o mesmo IV no site e no projeto. Em CBC, deve também ser escolhido o mesmo padding. Em CTR, a saída tem o mesmo comprimento da entrada e não é necessário preenchimento.

Modo CBC:
Texto em claro: teste123
Chave: 0f0e0d0c0b0a09080706050403020100
IV: 0f0e0d0c0b0a09080706050403020100

Padding utilizado:PKCS#7
Resultado do site e resultado do projeto: QFNvQ8K8eGOVP4gJjqh/VA==
Texto recuperado na decifragem: teste123



Modo CTR:
Texto em claro: alo teste
Chave: 0f0e0d0c0b0a09080706050403020100
IV: 0f0e0d0c0b0a09080706050403020100
Resultado do site e resultado do projeto: TKcO4e7W4KEW
Texto recuperado na decifragem: alo teste

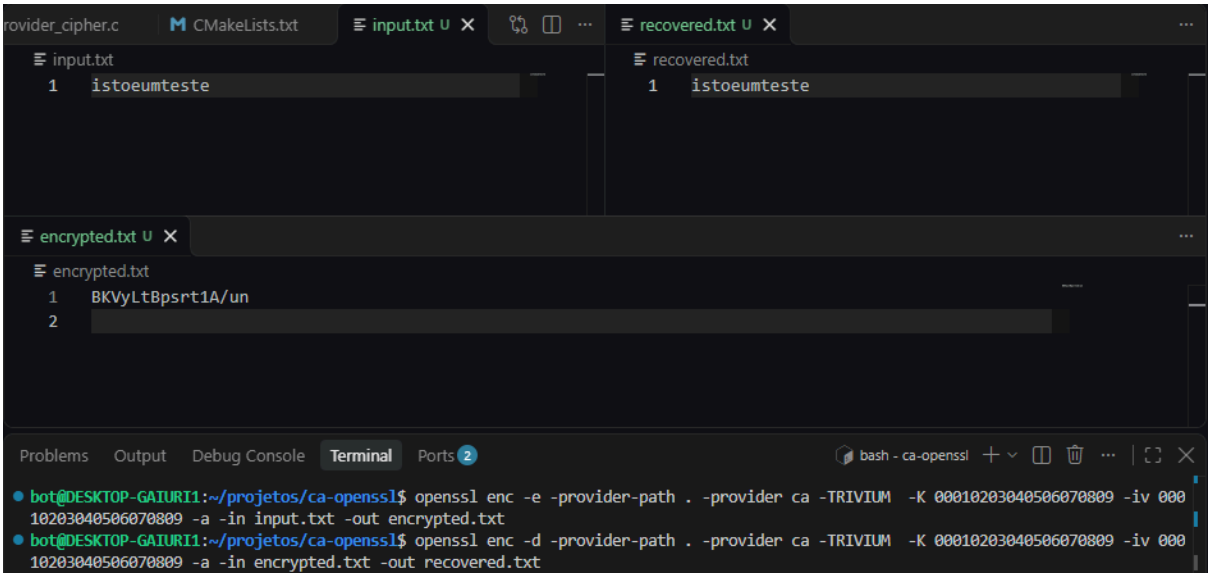


8.3. Respostas conhecidas do Trivium

A cifragem Trivium foi comparada com a implementação disponível no site Asecuritysite. Foram usados o mesmo texto, a mesma chave e o mesmo IV no site e no projeto.

O site apresentou o resultado em hexadecimal. Para o comparar com o ficheiro produzido pelo projeto, esse valor foi convertido para Base64 através do conversor Hex to Base64. Depois da conversão, os dois resultados coincidiram. A conversão para Base64 apenas permite apresentar os mesmos bytes num formato de texto que pode ser copiado e comparado.

Texto em claro utilizado: istoeumteste
Chave: 00010203040506070809
IV: 00010203040506070809
Resultado hexadecimal do site:04a5722ed069b2bb7503fba7
Resultado convertido para Base64: BKVyLtBpsrt1A/un
Resultado Base64 do projeto: BKVyLtBpsrt1A/un
Texto recuperado na decifragem: istoeumteste



8.4. Diferentes comprimentos e recuperação de ficheiros

Para verificar o funcionamento com diferentes tamanhos de entrada, foram testados os seis casos pedidos no enunciado: ficheiro vazio, ficheiro de 1 byte, um bloco de 16 bytes, um bloco mais 1 byte, vários blocos e um ficheiro grande. Em cada caso, foi feita a cifragem e a decifragem com as cifras e os modos implementados. O ficheiro recuperado foi comparado com o original através do comando *cmp* nativo do linux. A tabela seguinte apresenta os tamanhos utilizados, os tamanhos dos ficheiros cifrados e o resultado de cada comparação entre os ficheiros decifrados e originais.

Tamanhos de ficheiro a testar	Tamanhos de ficheiros cifrados (SPECK ECB/CBC)	Tamanhos de ficheiros cifrados (SPECK CTR / TRIVIUM)	Resultado da comparação (input - recovered)
0 bytes	16 bytes	0 bytes	Igual

1 byte	16 bytes	1 byte	Igual
16 bytes	32 bytes	16 bytes	Igual
1 bloco + 1 byte	32 bytes	17 bytes	Igual
vários blocos (512 bytes)	528 bytes	512 bytes	Igual
ficheiro grande (10 MiB - 10 485 760 bytes)	10 485 776 bytes	10 485 760 bytes	Igual

Todos estes testes foram realizados com a K = 0f0e0d0c0b0a09080706050403020100 e IV (quando aplicável) = 00000000000000000000000000000000

8.5. Fragmentação, contextos e erros

Foram realizados testes com os dados divididos por várias chamadas a **update**, para confirmar que a divisão da entrada não altera o resultado. No Trivium, uma sequência conhecida de 64 bytes foi dividida nas 65 posições possíveis. Foram também verificadas cópias de contexto durante uma operação, reinicialização, processamento com entrada e saída no mesmo buffer e acesso a buffers desalinhados nos núcleos.

Na integração com o OpenSSL, foram testados parâmetros com tamanhos ou tipos incorretos, inicializações sem os dados necessários e inicializações feitas por etapas. Verificou-se ainda a possibilidade de reutilizar a chave numa reinicialização e de continuar uma operação a partir de uma cópia do contexto.

Os testes de finalização incluíram padding inválido, texto cifrado truncado e entradas com blocos incompletos quando se usa -nopad. Foram também testados buffers de saída demasiado pequenos, a repetição da operação depois de disponibilizar espaço suficiente e chamadas a update após a finalização. A interface direta ***EVP_Cipher*** foi verificada separadamente.

Em CTR, foi testada a passagem do contador pelo seu limite e a recusa de continuar após o esgotamento. No Trivium, o contador de bytes foi colocado próximo do limite para verificar essa fronteira sem gerar toda a sequência. Confirmou-se que um pedido acima do limite é recusado sem alterar o estado nem a saída.

9. Avaliação de desempenho

9.1. Ambiente experimental

Para a comparação de desempenho foi utilizado, como pedido no enunciado, o comando **openssl speed**, comparando SPECK-ECB/CBC/CTR com AES-128-ECB/CBC/CTR, respetivamente, e Trivium com ChaCha20. Todas as medições são realizadas na mesma máquina e em condições de carga semelhantes. [1, 9, 10]

A máquina utilizada tem um processador AMD Ryzen 7 8845HS, de arquitetura x86_64, com 8 núcleos e 16 processadores lógicos. O sistema é Debian GNU/Linux 13.7, executado em WSL2.

O processador disponibiliza as extensões AES, VAES e AVX2. Foi mantida a deteção automática das capacidades do CPU pelo OpenSSL, sem definir a variável **OPENSSL_ia32cap** para restringir a aceleração disponível. As implementações SPECK e Trivium do projeto usam código C sem rotinas específicas de aceleração por hardware.

O provider foi compilado sem selecionar explicitamente Debug ou Release. Os ficheiros gerados pelo CMake indicam apenas a opção **-fPIC**, sem opções explícitas de otimização.

9.2. Procedimento de medição

openssl speed -evp mede o algoritmo através de **EVP**. Para as cifras do projeto carregam-se **default** e **ca**: default fornece, entre outros serviços, a geração aleatória necessária à preparação do benchmark. Restringir globalmente todas as pesquisas a provider=ca pode impedir a obtenção desses serviços. Os nomes SPECK128-128-* e TRIVIUM identificam as cifras do projeto.

Exemplo de uma medição, com tempo real decorrido e buffer de 1 024 bytes: [11]

```shell
openssl speed -provider-path . -provider default -provider ca -elapsed -seconds
3 -bytes 1024 -evp SPECK128-128-CTR

openssl speed -provider-path . -provider default -provider ca -elapsed -seconds
3 -bytes 1024 -evp TRIVIUM

openssl speed -provider default -elapsed -seconds 3 -bytes 1024 -evp
AES-128-CTR

openssl speed -provider default -elapsed -seconds 3 -bytes 1024 -evp ChaCha20
```

O procedimento proposto repete cada algoritmo para buffers de 16, 64, 256, 1 024 e 8 192 bytes, com pelo menos três execuções por configuração e a mediana como resultado. Os algoritmos ECB e CBC seguem o mesmo modelo de comando. Para medir a decifragem acrescenta-se ***-decrypt*** e conserva-se o resto da configuração. Deve ser registada a dispersão, por exemplo mínimo e máximo, e guardada a saída original de cada execução.

9.3. Resultados a preencher

Cada medição teve uma duração de três segundos, usando o tempo decorrido através da opção **-elapsed**. Os resultados foram convertidos para MB/s. As tabelas representam Cifragem e Decifragem respetivamente.

Cifra	16 bytes	64 bytes	256 bytes	1024 bytes	8192 bytes
SPECK128-128-ECB	27.260	30.142	34.523	51.498	30.141
SPECK128-128-CBC	27.118	31.743	32.199	22.889	32.053

SPECK128-128-CTR	26.926	24.331	30.837	31.839	25.676
Trivium	0.208	0.235	0.179	0.232	0.235
AES-128-ECB	619.988	2227.895	5947.513	6770.828	6300.344
AES-128-CBC	589.325	824.087	844.684	760.527	895.768
AES-128-CTR	528.561	1606.845	3270.804	5105.782	4759.514
ChaCha20	225.381	346.772	1338.261	2139.398	3174.610

Cifra	16 bytes	64 bytes	256 bytes	1024 bytes	8192 bytes
SPECK128-128-ECB	29.318	30.057	24.034	31.450	29.554
SPECK128-128-CBC	22.419	31.259	30.599	22.764	31.547
SPECK128-128-CTR	27.069	22.411	31.878	32.721	28.639
Trivium	0.211	0.241	0.236	0.202	0.237
AES-128-ECB	636.868	2171.585	4965.709	6654.668	6148.355
AES-128-CBC	545.957	2235.953	4450.355	5335.732	7025.943
AES-128-CTR	411.073	1515.596	3564.476	4847.155	4765.005
ChaCha20	221.534	338.391	993.797	2955.615	3237.877

9.4. Discussão

Os testes de validação confirmaram o funcionamento das cifras e dos modos implementados. No entanto, e como esperado, o desempenho ficou muito abaixo das implementações AES e ChaCha20 do OpenSSL, sobretudo no Trivium, devido ao deslocamento do estado bit a bit. O código privilegia a simplicidade de leitura e foi compilado sem opções explícitas de otimização. Os resultados mostram, por isso, as limitações de desempenho desta implementação e não dos algoritmos.

10. Problemas encontrados e soluções

No Trivium, foi necessário corrigir a ordem de leitura dos bits da chave e do IV para obter os resultados dos vetores escolhidos. As equações do algoritmo mantiveram-se, mudou apenas a conversão entre bytes e bits. Para comparar com a implementação de Wu Han[4] tida como referência, foi também necessário inverter a ordem dos bytes da chave e do IV fornecidos à referência.

Na integração com o OpenSSL, o ECB passou a aceitar um ponteiro IV com comprimento zero, pois este modo não utiliza IV.

Para executar **openssl speed**, foi necessário carregar também o provider **default**, que fornece o gerador aleatório usado na preparação do benchmark.

Foram ainda corrigidos dois problemas de gestão dos buffers. Em **block_update,** a entrada é copiada quando existem bytes pendentes e a saída ocupa o mesmo buffer, evitando sobrescrever dados ainda por ler. Na finalização CBC, um contexto temporário permite verificar o espaço disponível antes de alterar o estado da operação.

Após as alterações, foram repetidos os testes com vetores conhecidos, os testes através de EVP e os ciclos de cifragem e decifragem de ficheiros, sem falhas detetadas.

11. Limitações e possíveis melhorias

As cifras implementadas não garantem a integridade nem a origem dos ficheiros. Uma alteração no texto cifrado ou uma chave errada pode produzir dados incorretos sem causar erro. Em CTR e Trivium, reutilizar a mesma chave e IV compromete a segurança. A gestão destes valores fica a cargo do utilizador.

O projeto também não guarda automaticamente os parâmetros necessários à decifragem. É preciso conhecer o algoritmo, a chave, o IV e o padding utilizados, seria uma boa ideia conseguir ler ficheiros e receber por aí uma chave.

Como melhorias, seria possível compactar o estado do Trivium e otimizar o processamento.

12. AI usage statement, referências de código e entrega

Utilização de inteligência artificial

Foi utilizado o Codex como apoio à pesquisa de referências, implementação e explicação do código, identificação de erros, validação, preparação do rascunho do relatório e redação completa da secção de referências. As decisões sobre a implementação e a responsabilidade pelo conteúdo entregue em código e relatório final é completamente dos autores.

Referências do código

A implementação do SPECK foi adaptada de um guia encontrado online. A atualização do estado do Trivium foi adaptada da implementação C de Wu Han, sob licença MIT. Os vetores de teste foram obtidos da coleção ECRYPT indicada nas referências. O código de integração com o OpenSSL foi desenvolvido para este projeto com o apoio referido anteriormente. [2, 4, 5]
As fontes utilizadas estão identificadas no relatório e no código.

Entrega

A entrega reúne o código das cifras e do provider, o ficheiro CMakeLists.txt, e o relatório final.

13. Conclusão

O projeto integrou SPECK128/128 nos modos ECB, CBC e CTR, e Trivium no OpenSSL 3 através de um provider, permitindo cifrar e decifrar ficheiros com os comandos existentes, sem alterar o executável.

Os testes realizados confirmaram o funcionamento das cifras e a recuperação de ficheiros com diferentes tamanhos. As medições mostraram um desempenho bastante inferior ao das implementações do OpenSSL, sobretudo no Trivium. A implementação cumpriu os objetivos funcionais, priorizando bastante a apresentação e legibilidade do código, logo tem margem significativa para otimização.

14. Referências

[1] Enunciado da unidade curricular. Cybersecurity MSc: Applied Cryptography 2025–26. 1st Project: Extending the OpenSSL command-line tool with new block and stream ciphers through an OpenSSL 3 provider. Versão 1.0, 28 de setembro de 2026.

[2] Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B., Wingers, L. SIMON and SPECK Implementation Guide. 15 de janeiro de 2019. Secções de conversão de bytes, SPECK128/128 e vetores de teste.
https://nsacyber.github.io/simon-speck/implementations/ImplementationGuide1.1.pdf

[3] De Cannière, C., Preneel, B. Trivium Specifications. ECRYPT Stream Cipher Project.
https://cr.yp.to/streamciphers/trivium/desc.pdf . Descrição adicional: De Cannière, C. Analysis and Design of Symmetric Encryption Algorithms. Tese de doutoramento, KU Leuven, maio de 2007, secção 6.7. https://cosicdatabase.esat.kuleuven.be/backend/publications/files/these/139

[4] Wu Han. trivium-c/trivium.c. Repositório wuhanstudio/trivium, commit b8bc30f412c4c9cf3d8ea0c7ed1755be577ebdd1.
https://github.com/wuhanstudio/trivium/blob/b8bc30f412c4c9cf3d8ea0c7ed1755be577ebdd1/trivium-c/trivium.c

[5] ECRYPT Stream Cipher Project. Trivium 80/80 test vectors. Cópia preservada em cantora/avr-crypto-lib, ficheiro testvectors/trivium-80.80.test-vectors, blob d2784589582019ee1e87b68680edd6e0b4d8ef9c.
https://github.com/cantora/avr-crypto-lib/blob/master/testvectors/trivium-80.80.test-vectors

[6] OpenSSL Project. provider-base, documentação OpenSSL 3.5.
https://docs.openssl.org/3.5/man7/provider-base/

[7] OpenSSL Project. provider-cipher, documentação OpenSSL 3.5.
https://docs.openssl.org/3.5/man7/provider-cipher/

[8] OpenSSL Project. openssl-enc, documentação OpenSSL 3.5.
https://docs.openssl.org/3.5/man1/openssl-enc/

[9] OpenSSL Project. EVP_CIPHER-AES, documentação OpenSSL 3.5. https://docs.openssl.org/3.5/man7/EVP_CIPHER-AES/

[10] OpenSSL Project. EVP_CIPHER-CHACHA, documentação OpenSSL 3.5. https://docs.openssl.org/3.5/man7/EVP_CIPHER-CHACHA/

[11] OpenSSL Project. openssl-speed, documentação OpenSSL 3.5. https://docs.openssl.org/3.5/man1/openssl-speed/